

Modul mahasiswa Lab 04 - Virtualisasi dan Container

Kebijakan kelas: Lab ini latihan formatif, tanpa tugas, nilai, atau penyerahan terpisah. Satu proyek besar dikerjakan oleh kelompok **3 orang**, dengan presentasi checkpoint minggu 7 (UTS) dan hasil akhir minggu 14 (UAS). Simpan hasil lab hanya bila berguna sebagai referensi atau bukti proses proyek. Baca [brief proyek kelompok](#). Bobot resmi tetap mengikuti RPS/LMS.

Cara membaca gambar: foto Docker Desktop dan Chrome adalah screenshot langsung dari praktik. Kartu terminal menunjukkan keluaran command yang benar-benar dijalankan dan ditata ulang agar terbaca. Setiap langkah bernomor memiliki gambar hasil; jika ingin menyimpan catatan opsional, ambil screenshot milik Anda setelah menjalankan perintah di dekat gambar.

COMP6991031 | Praktikum kasus kerja harian | Windows PowerShell utama, Bash/Linux alternatif.

Modul ini memuat **seluruh kunci challenge A-E**. Jalankan perintah dari root repo, kecuali bila langkah menyebut direktori lain. Setiap gambar adalah contoh hasil uji di komputer dosen; IP, waktu, ID, dan versi patch bisa berbeda. Simpan screenshot hasil praktikmu sendiri bila ingin menggunakannya sebagai referensi proyek kelompok.

Hasil belajar dan kasus

Di tim operasi, halaman status internal harus tersedia saat ada insiden. Kamu menangani tiga kebutuhan: melihat kondisi layanan, memperbaiki halaman status tanpa membangun image baru, dan menyiapkan instance kedua ketika port utama sudah dipakai. Setelah lab, kamu dapat menjelaskan VM vs container, hubungan Docker client-daemon-registry, image vs container, namespaces/cgroups, port, bind mount, log, dan siklus `run -> stop -> start -> rm`.

Komponen	Arti pada kasus ini
Image <code>nginx:alpine</code>	Paket template server web yang tetap sama saat teks halaman berubah.
Container <code>cloudlab-site</code>	Proses Nginx yang berjalan dari image; dapat dihentikan dan dimulai ulang.
<code>site/index.html</code>	File di laptop yang dibaca Nginx melalui bind mount read-only.
<code>127.0.0.1:8088:80</code>	Port 8088 di laptop diteruskan ke port 80 dalam container, hanya pada loopback laptop.
Container <code>cloudlab-canary</code>	Instance kedua pada port host 8089 saat 8088 masih dipakai.

VM membawa guest OS sendiri di atas hypervisor. Container berbagi kernel host dan diisolasi dengan namespaces, sedangkan cgroups membatasi sumber daya. Docker CLI mengirim perintah ke daemon; daemon mengambil image dari registry lalu membuat container. Lab ini memakai container Linux melalui Docker Desktop/Engine, tanpa memasang hypervisor kedua.

0. Mulai dari komputer kampus dan clone repo

Jalur utama Lab 04 memakai **Windows PowerShell** dan Docker Desktop dengan **Linux Engine** yang sudah disiapkan kampus. Gunakan akun serta folder kerja pribadi yang dapat ditulis; repo boleh diletakkan di folder mana pun, termasuk path dengan spasi. Clone repo publik pengajar sudah cukup untuk mengikuti praktik formatif; tidak ada kewajiban membuat repo, push, atau laporan per lab. Lab ini menggunakan `docker pull` dan `docker run` untuk image yang tersedia, serta file HTML host melalui bind mount.

0.1 Periksa Git dan Docker Desktop

Buka **Docker Desktop** melalui Start, tunggu **Engine running**, kemudian buka PowerShell biasa:

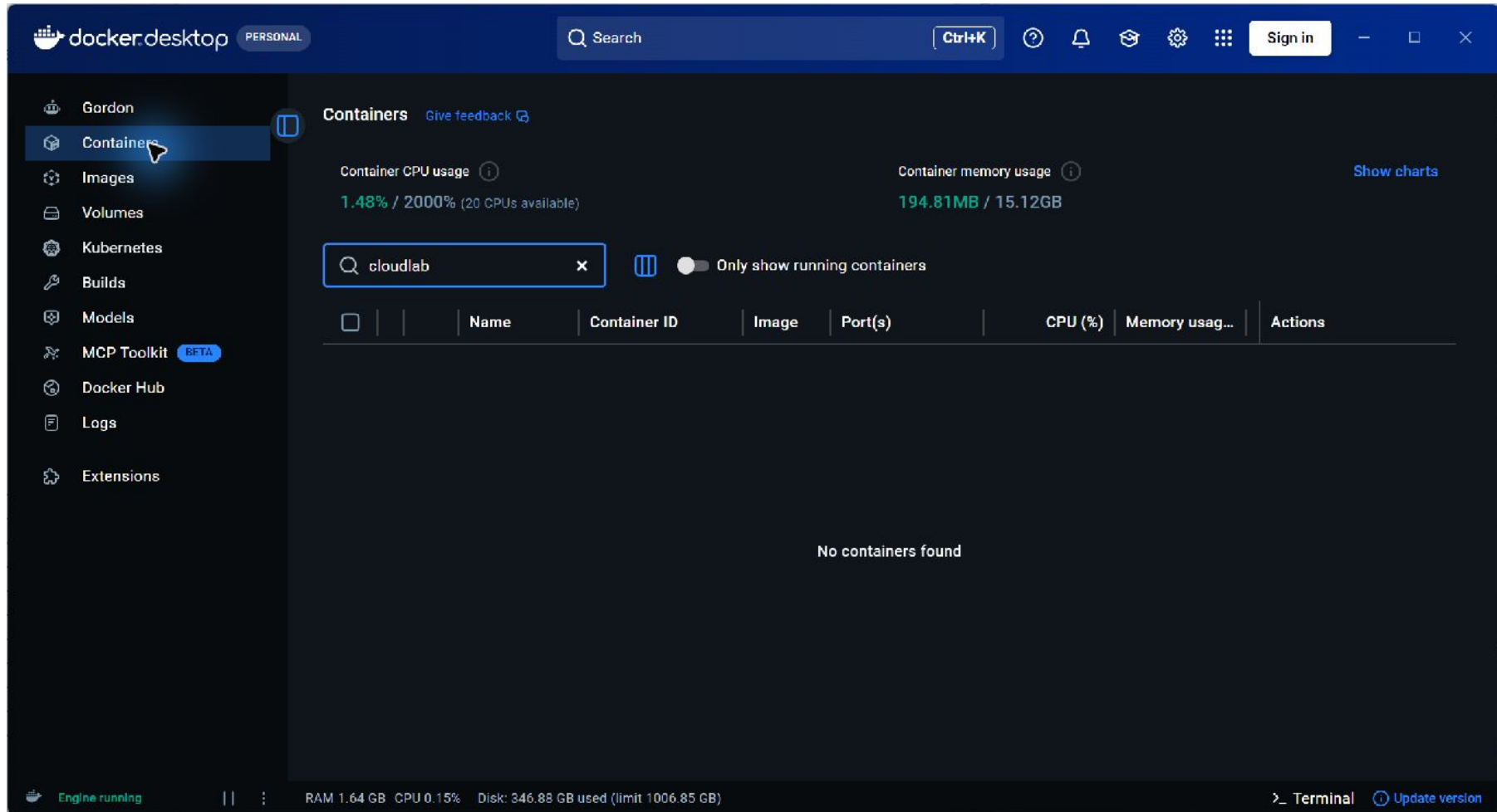
```
git --version
docker version
docker info --format '{{.OSType}}'
docker ps
```

Cara membaca: Git menampilkan versi; `docker version` menunjukkan **Client dan Server**; `docker info --format` harus mencetak **linux**. Client tanpa Server berarti daemon belum siap. `docker ps` membantu memeriksa nama container dan port **8088/8089** yang dipakai lab. Lihat [kendala port](#) sebelum memakai port yang sudah dimiliki proses lain.

Jika Git/Docker belum tersedia, Engine tidak dapat dimulai, atau instalasi/virtualisasi dibatasi kebijakan PC, minta bantuan pengelola kampus. Instalasi/perubahan sistem mengikuti kebijakan pengelola. Alternatif kelas adalah **Code** → **Codespaces** di GitHub, menggunakan lingkungan Linux dengan daemon Docker yang siap; ikuti blok Bash pada modul dan URL tab Ports untuk browser. [Docker Desktop Windows](#)

Gambar 1. Docker Desktop siap untuk Lab 04

```
docker version
```



Langkah: buka Docker Desktop dan jalankan `docker version`. Fungsi: memastikan Engine aktif sebelum menarik image. Cara kerja: CLI bertanya kepada daemon Docker; tanpa daemon, semua langkah berikutnya gagal. Baca hasil: indikator Engine running dan versi client/server tampil.

0.2 Clone materi Lab 04 ke folder pilihan Anda

Jika folder `meet4CloudService` sudah ada pada PC, gunakan bagian **0.3**. Untuk clone pertama, buka [repo Lab 04](#), pilih **Code** → **Local** → **HTTPS**. Contoh berikut memakai Documents; Anda dapat memilih folder pribadi lain yang dapat ditulis:

```
$campusFolder = Join-Path ([Environment]::GetFolderPath('MyDocuments')) 'CloudServices'
New-Item -ItemType Directory -Path $campusFolder -Force | Out-Null
Set-Location -LiteralPath $campusFolder
git clone https://github.com/SeedFlora/meet4CloudService.git
Set-Location -LiteralPath 'meet4CloudService'
Get-Location
Get-ChildItem -LiteralPath 'site/index.html', 'scripts/start-lab04.ps1', 'tests/challenge.ps1', `
    'tests/challenge.sh'
git remote -v
```

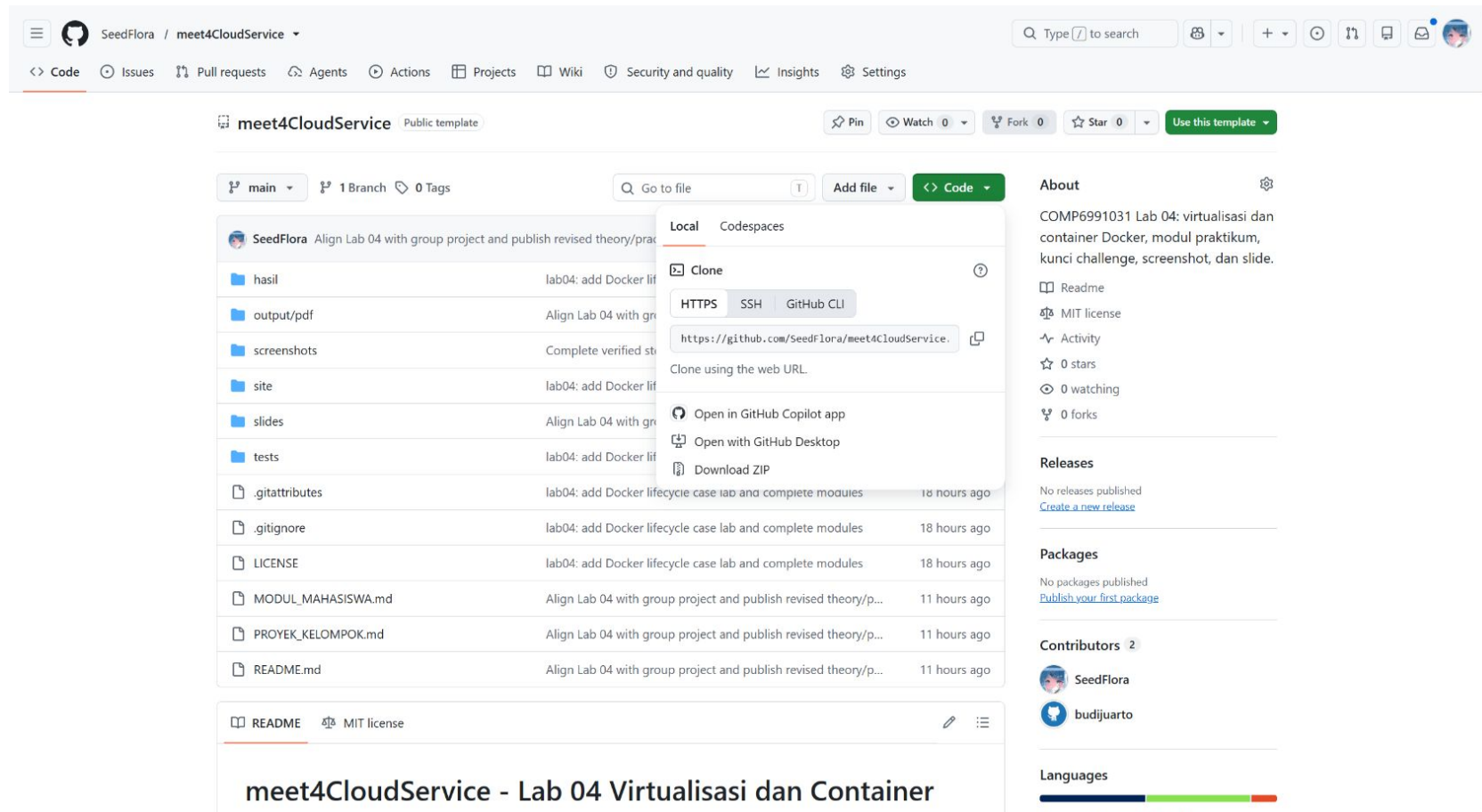
Fungsi/cara kerja: `GetFolderPath` mengambil Documents milik akun, termasuk bila lokasinya dialihkan; `New-Item` membuat folder CloudServices tanpa menghapus isinya. `Set-Location` memindahkan terminal. `git clone` menyalin kode/riwayat Git dan membuat folder repo beserta `.git` dan remote `origin`.

Checkpoint: empat file di atas ditemukan dan origin menuju SeedFlora/meet4CloudService. Pada contoh ini lokasi berakhir pada `CloudServices\meet4CloudService`; pada PC Anda path boleh berbeda. Folder yang langsung berisi `site/`, `scripts/`, dan `tests/` adalah **root repo** untuk perintah manual Lab 04. Jika Documents tidak dapat ditulis, gunakan folder pribadi yang diizinkan pengelola.

Gambar 2. Menu HTTPS GitHub untuk clone materi Lab 04

Code > Local > HTTPS

```
git clone https://github.com/SeedFlora/meet4CloudService.git
```



Command / langkah: Code > Local > HTTPS dan `git clone https://github.com/SeedFlora/meet4CloudService.git`. Fungsi/cara kerja: ambil URL sumber dari GitHub lalu salin kode/riwayat ke repo lokal dengan metadata .git dan origin. Baca: gambar menu repo pengajar memperlihatkan pilihan HTTPS; URL lengkap tersedia pada command agar dapat disalin. Gambar ini menu GitHub, bukan hasil PowerShell PC kampus; validasi lokasi dan file setelah clone pada PC Anda.

Hasil clone sudah mempunyai metadata Git, sehingga **tidak perlu** `git init`. Remote publik ini milik pengajar; **jangan push ke repo pengajar**. Perubahan latihan dapat tetap lokal dan tidak wajib dikumpulkan. [Panduan clone GitHub](#)

0.3 Jika repo sudah ada

Masuk folder repo yang sama melalui **VS Code** → **File** → **Open Folder**, kemudian **Terminal** → **New Terminal**. Periksa lokasi, remote, dan perubahan dahulu:

```
Get-Location
Test-Path -LiteralPath '.\site\index.html' -PathType Leaf
git remote -v
git status --short
```

`Test-Path` harus **True** dan origin harus repo yang dimaksud. **Hanya jika** `git status --short` **kosong**, jalankan:

```
git pull --ff-only
```

`--ff-only` menerima pembaruan fast-forward dan berhenti bila history berbeda. Jika ada perubahan lokal, tinjau/simpan pekerjaan Anda atau gunakan folder baru; jangan overwrite, reset paksa, atau menghapus file untuk memaksa update. Pada PC bersama, periksa bahwa folder/repo memang milik sesi Anda.

0.4 Repo pribadi/kelompok opsional dan kebutuhan internet

Untuk menyimpan kontribusi proyek kelompok, pilih **Use this template** → **Create a new repository** pada GitHub dan tentukan owner Anda/kelompok. Ambil URL **Code** → **Local** → **HTTPS** dari repo baru, lalu clone URL milik Anda ke folder baru; periksa `git remote -v` sebelum push. Proyek tetap kelompok **3 orang**, dipresentasikan minggu **7 (UTS)** dan **14 (UAS)**; tidak ada penyerahan/repo wajib per lab. [Template GitHub](#)

Lokal bukan sepenuhnya offline: clone memerlukan internet, dan `docker pull` pertama mengunduh image dari registry. Setelah image tersedia, praktik Nginx/HTML melalui localhost dapat dilakukan tanpa internet. Pull/update image baru tetap memerlukan koneksi. Lanjutkan bagian 1 dari root repo yang baru dibuka. Jika memakai Linux/Codespaces, gunakan blok Bash karena sintaks bind mount berbeda; browser Codespaces menggunakan URL Ports, bukan localhost laptop. [Docker pull](#), [publikasi port](#)

0.5 Pilih urutan praktik manual atau shortcut situs

Praktik lengkap: ikuti bagian 1 untuk image/environment, bagian 2 untuk lifecycle Nginx pertama, stop/remove container pertama, lalu bagian 3 untuk deploy situs. Pada bagian 3 Anda dapat memilih command manual atau starter sebagai shortcut/repair. Setelah itu lanjutkan bagian 4 untuk outage, hotfix, dan canary.

Jika sudah memakai starter dan cloudlab-site masih Up pada 8088, jangan membuat cloudlab-nginx bagian 2 pada port yang sama dan jangan menjalankan docker run situs kedua. Lanjutkan bagian 4 untuk operasi pada situs yang hidup; bagian 1 tetap dapat dicoba untuk image/environment. Anda dapat meninjau penjelasan lifecycle bagian 2 tanpa menjalankan deploy yang memakai port tersebut.

Starter menghitung root repo dari **lokasi file script**, bukan folder aktif terminal; perintahnya ada pada bagian **3.1**. Ia tidak memindahkan working directory terminal pemanggil. Untuk perintah manual/hotfix berikutnya, buka **folder repo sebenarnya** di VS Code dan buat terminal baru. Contoh struktur berikut membantu membedakan folder luar dan repo:

```

folder-kerja/
  site/                <- folder luar, jangan dipakai sebagai mount lab
  meet4CloudService/  <- root repo yang dipilih pada Open Folder
    scripts/start-lab04.ps1
    scripts/start-lab04.sh
    site/index.html     <- file yang harus terlihat di container
    tests/challenge.ps1
    tests/challenge.sh

```

1. Kenali tiga image dan environment variable

```

docker pull nginx:alpine
docker pull python:3.12-alpine
docker pull node:22-alpine
docker image ls nginx
docker image ls python
docker image ls node
docker image inspect nginx:alpine --format '{{.Os}}/{{.Architecture}} {{.Id}}'
docker run --rm python:3.12-alpine python --version
docker run --rm node:22-alpine node --version
docker run --rm -e CLASS_SECTION=cloud python:3.12-alpine sh -c 'echo "$CLASS_SECTION"'

```

`pull` menyimpan image dari registry ke mesin. `image ls/inspect` membaca metadata image; ID akan tetap sama ketika hanya file `site/` yang berubah. `run --rm` membuat container sekali pakai dan menghapusnya saat selesai. Opsi `-e` menyuntik environment variable ke proses dalam container; baris terakhir harus mencetak `cloud`. Versi patch dan image ID bisa berubah sesuai tanggal pull.

Gambar 3. Image dan proses sekali pakai telah diverifikasi

```
docker pull
image ls
-e CLASS_SECTION=cloud
```

A • image dan environment

```
PS> docker version --format 'Client / Server'
Client 29.6.2 | Server 29.6.2
PS> docker image ls nginx
nginx:alpine          df221db836e1          94.4MB          27.2MB          U
PS> docker image ls python
python:3.12-alpine    1b668429b351          83.6MB          21.3MB
PS> docker image ls node
node:22-alpine        0a7108bf6c7b          238MB           61.1MB
PS> docker run --rm python:3.12-alpine python --version
Python 3.12.15
PS> docker run --rm node:22-alpine node --version
v22.23.3
PS> docker run --rm -e CLASS_SECTION=cloud python:3.12-alpine sh -c 'echo "$CLASS_SECTION"'
cloud
```

Cuplikan output uji lokal aktual; baris/kolom dipilih dan ditata agar terbaca.

Langkah: jalankan tiga `docker pull`, `image ls`, Python/Node versi, dan contoh `-e CLASS_SECTION=cloud`. Fungsi: membedakan image tersimpan dari proses container sekali pakai. Cara kerja: proses `sh` dalam image Python membaca environment container lalu mencetak `cloud`; `--rm` membersihkan container setelah proses keluar. Baca hasil: ketiga image ada, versi muncul, dan output `cloud`. Gambar adalah cuplikan output uji lokal aktual yang ditata agar terbaca.

Sebelum membuat situs, jalankan & `.\tests\challenge.ps1` (PowerShell) atau `bash tests/challenge.sh` (dari Git Bash/Linux) sekali untuk melihat checklist kondisi layanan. **FAIL pada starter memang diharapkan**. Setelah tiga image tersedia, contoh lokal memberi 3 `PASS`, 11 `FAIL`; jumlah awal dapat berbeda bila image belum ditarik. Baca hint setelah FAIL sebagai urutan pekerjaan, lalu ulangi checker di akhir.

Gambar 4. Challenge starter menunjukkan pekerjaan yang belum selesai

jalankan checker setelah menarik image tetapi sebelum deploy situs.

●●● Challenge pada starter · petunjuk pekerjaan

```
PS> & .\tests\challenge.ps1
PASS A image nginx:alpine tersedia
PASS A image python:3.12-alpine tersedia
PASS A image node:22-alpine tersedia
FAIL B cloudlab-site berjalan dari nginx:alpine
FAIL B port 8088 hanya di 127.0.0.1
FAIL B folder site/ di-bind mount read-only
FAIL C halaman utama HTTP 200 di 8088
FAIL D halaman menampilkan INCIDENT-042: Pemeliharaan selesai
FAIL E cloudlab-canary berjalan dari nginx:alpine
Challenge: 3 PASS, 11 FAIL
```

Cuplikan output uji lokal aktual; baris/kolom dipilih dan ditata agar terbaca.

Langkah: jalankan checker setelah menarik image tetapi sebelum deploy situs. Fungsi: memetakan syarat challenge A-E. Cara kerja: checker membaca image, metadata Docker, dan HTTP lokal tanpa membuat container. Baca hasil: A sudah PASS, B-E masih FAIL karena situs, hotfix, dan canary belum dikerjakan; ini kondisi awal yang benar. Gambar adalah cuplikan output uji lokal aktual yang ditata agar terbaca.

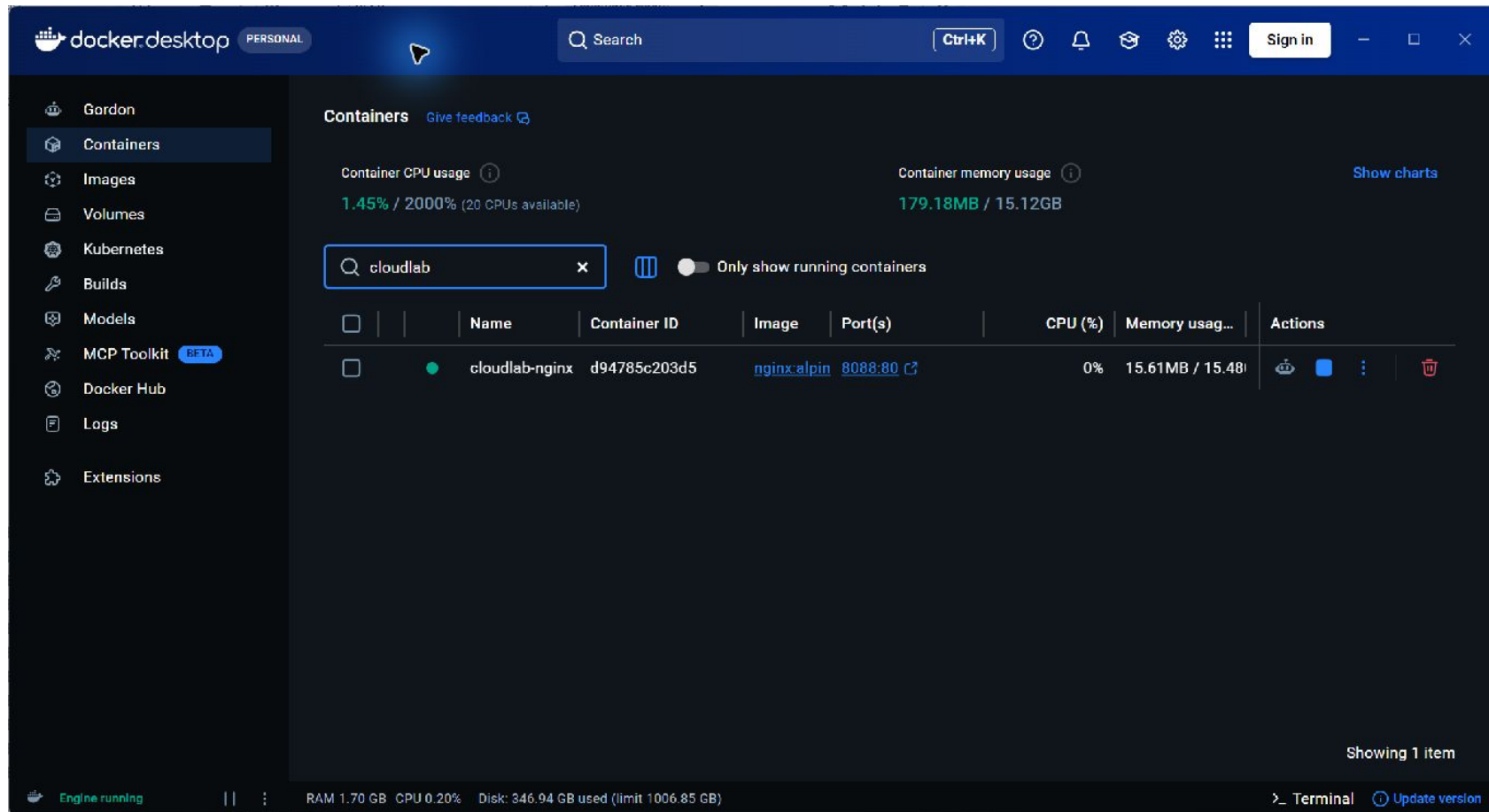
2. Operasikan container Nginx pertama

```
docker run --name cloudlab-nginx -d -p 127.0.0.1:8088:80 nginx:alpine
docker ps --filter name=cloudlab-nginx
curl.exe -I http://127.0.0.1:8088/
docker logs --tail 10 cloudlab-nginx
docker exec cloudlab-nginx sh -c 'hostname; ls /usr/share/nginx/html'
```

`run` membuat sekaligus memulai container. `-d` melepas proses dari terminal. `-p` meneruskan HTTP dari port laptop 8088 ke port container 80. `ps` menampilkan container hidup; `logs` membantu melihat request dan error; `exec` masuk untuk menjalankan perintah diagnostik tanpa mengubah image. `curl.exe -I` meminta header dan harus memberi HTTP 200. Buka juga `<http://127.0.0.1:8088/>` di Chrome: ini halaman bawaan Nginx, belum halaman kasus.

Gambar 5. Nginx hidup pada port 8088

```
docker run
docker ps
curl.exe -I
```



Langkah: jalankan `docker run`, `docker ps`, dan `curl.exe -I`. Fungsi: membuktikan proses web dan pemetaan port. Cara kerja: Docker mengikat `127.0.0.1:8088` ke `80/tcp` dalam container. Baca hasil: container Up, mapping 8088 ke 80, dan HTTP 200.

Sekarang amati siklus container yang **sama**:

```
docker stop cloudlab-nginx
docker ps --filter name=cloudlab-nginx
docker ps -a --filter name=cloudlab-nginx
docker start cloudlab-nginx
docker ps --filter name=cloudlab-nginx
docker stop cloudlab-nginx
docker rm cloudlab-nginx
```

Setelah `stop`, `docker ps` tidak menampilkan container, tetapi `ps -a` menampilkan `Exited`. `start` memakai container lama dan mempertahankan identitasnya; `rm` menghapus objek container setelah berhenti. Image `nginx:alpine` tetap ada. Langkah akhir membebaskan port 8088 untuk situs kasus.

Checkpoint sebelum bagian 3: `cloudlab-nginx` sudah di-stop dan di-remove pada dua command terakhir. Periksa `docker ps -a --filter name=cloudlab-`; container pertama tidak lagi tercantum. Jika masih Up, port 8088 masih dimilikinya. Jalankan `docker stop cloudlab-nginx` lalu `docker rm cloudlab-nginx` **hanya untuk container latihan milik Anda** sebelum membuat situs. Container lain yang memakai 8088 ditangani melalui bagian kendala.

Gambar 6. Status Exited dan Up dalam siklus container

```
stop
ps
ps -a
start
```

Siklus container • ID tetap saat stop/start

```
PS> docker stop cloudlab-nginx
PS> docker ps -a --filter name=cloudlab-nginx
d94785c203d5  nginx:alpine  Exited (0)  cloudlab-nginx
PS> docker start cloudlab-nginx
PS> docker ps --filter name=cloudlab-nginx
d94785c203d5  nginx:alpine  Up  127.0.0.1:8088->80/tcp
PS> docker stop cloudlab-nginx # kedua kali, sebelum rm
PS> docker rm cloudlab-nginx
```

Cuplikan output uji lokal aktual; baris/kolom dipilih dan ditata agar terbaca.

Langkah: **stop**, bandingkan **ps** dengan **ps -a**, lalu **start**. Fungsi: melatih diagnosis layanan berhenti. Cara kerja: Docker mengubah state proses container tanpa menghapus image atau membuat container baru. Baca hasil: ID sama, **Exited** sesudah stop dan **Up** sesudah start; **rm** dilakukan setelah stop kedua. Gambar adalah cuplikan output uji lokal aktual yang ditata agar terbaca.

3. Deploy halaman status dengan bind mount read-only

Bagian ini menunjukkan command manual di balik starter. Jika situs sudah dibuat dengan starter, gunakan pemeriksaan mount/HTTP di bawah tanpa menjalankan `docker run` kedua. Jika ingin mengulang deploy, starter dengan mode repair menangani container lab yang dikenali.

3.1 Shortcut atau repair dengan starter

Untuk urutan praktik lengkap, selesaikan lifecycle bagian 2 dan stop/remove `cloudlab-nginx` terlebih dahulu. Starter membantu menyalakan situs atau memperbaiki mount yang salah tanpa bergantung pada folder aktif terminal atau variabel `$mount` lama. Pilih **satu** command yang cocok:

PowerShell dari root repo:

```
& .\scripts\start-lab04.ps1 -Repair
```

PowerShell dari folder induk yang berisi meet4CloudService:

```
& .\meet4CloudService\scripts\start-lab04.ps1 -Repair
```

Dari lokasi lain: pilih file `scripts/start-lab04.ps1` di Explorer/VS Code, salin path lengkap, lalu panggil dengan `&` dan tanda kutip: `& 'PATH LENGKAP_REPO_ANDA\scripts\start-lab04.ps1' -Repair`. Ganti placeholder dengan path file yang benar. Tanda kutip membuat path dengan spasi menjadi satu argumen; tidak ada keharusan menggunakan drive atau nama folder tertentu.

Git Bash/Linux/Codespaces dari root repo:

```
bash scripts/start-lab04.sh --repair
```

Dari folder induk gunakan `bash "meet4CloudService/scripts/start-lab04.sh" --repair`. Dari lokasi lain panggil path lengkap file script dalam tanda kutip. Git Bash digunakan melalui **jendela Git Bash**, sedangkan terminal Codespaces adalah Linux.

Fungsi/cara kerja: starter memvalidasi repo, file `site/index.html`, dan Docker sebelum perubahan. Tanpa opsi repair, starter menolak nama container yang sudah ada. `-Repair/--repair` mengganti **hanya** container `cloudlab-site/cloudlab-nginx` dengan image Nginx yang dikenali; tidak menghapus container lain atau nama lab yang memakai image berbeda. File HTML pada host tidak dihapus. Port default 8088 diteruskan ke 80 dalam container. Jika 8088 dipakai proses lain, gunakan `-SitePort 8090` atau `--port 8090`, buka URL 8090, dan catat bahwa checker standar memakai 8088/8089. Jangan menghentikan proses milik orang lain.

Checkpoint: Source/mount menunjuk folder **site di repo yang berisi index.html**, container Up, dan HTTP 200. Di laptop buka `<http://127.0.0.1:8088/>`; di Codespaces buka URL **Ports 8088**. Jika starter menolak prasyarat, baca error dan perbaiki path/Engine sebelum mengulang. Starter menyalakan situs; ia belum menjalankan seluruh challenge A-E.

Sesudah starter berhasil: lewati `docker run` manual bagian 3.2 dan pembuatan `cloudlab-nginx` pada 8088. Periksa mount/HTTP lalu lanjutkan bagian 4. Starter tidak mengubah working directory pemanggil; untuk hotfix HTML manual, buka root repo dan terminal baru seperti langkah berikut.

3.2 Deploy manual setelah validasi lokasi dan port

Pada VS Code pilih **File** → **Open Folder** → **folder repo sebenarnya**, lalu **Terminal** → **New Terminal**. Folder yang dipilih langsung berisi `site/index.html`, `scripts/`, dan `tests/`. Jika workspace luar mempunyai subfolder `meet4CloudService`, pilih subfolder tersebut; `site/` luar dapat berbeda atau kosong. **Jangan membuat folder site kosong untuk menghilangkan error**. Selesaikan stop/remove `cloudlab-nginx` pada bagian 2 terlebih dahulu agar 8088 tersedia.

Jalankan **seluruh** blok sesuai terminal. PowerShell `$siteDir` dan `$mount` harus dibuat pada **sesi terminal yang sama** dengan `docker run`; variabel dari terminal lama tidak otomatis ada pada terminal baru. Blok memakai scope sendiri agar validasi gagal menghentikan deploy. Jangan menyalin baris `docker run` saja.

Windows PowerShell:

```
& {
  Get-Location
  if (-not (Test-Path -LiteralPath '.\site\index.html' -PathType Leaf)) {
    throw 'STOP: buka root repo yang berisi site/index.html, lalu ulangi seluruh blok.'
  }
  $siteDir = (Resolve-Path -LiteralPath '.\site' -ErrorAction Stop).Path
  $mount = "type=bind,source=$siteDir,target=/usr/share/nginx/html,readonly"
  Write-Output $mount
  docker run --name cloudlab-site -d -p 127.0.0.1:8088:80 --mount "$mount" nginx:alpine
  if ($LASTEXITCODE -ne 0) { throw 'Docker run gagal; baca error sebelum melanjutkan.' }
  docker ps --filter name=cloudlab-site
  docker port cloudlab-site 80
  curl.exe -I http://127.0.0.1:8088/
}
```

Linux/Codespaces Bash:

```
(
  pwd
  test -f "site/index.html" || { echo "STOP: buka root repo yang berisi site/index.html"; exit 1; }
  site_dir="$(cd site && pwd)" || exit 1
  mount="type=bind,source=$site_dir,target=/usr/share/nginx/html,readonly"
  printf '%s\n' "$mount"
  docker run --name cloudlab-site -d -p 127.0.0.1:8088:80 --mount "$mount" nginx:alpine || exit 1
  docker ps --filter name=cloudlab-site
  docker port cloudlab-site 80
  curl -I http://127.0.0.1:8088/
)
```

Path `source` menunjuk folder **host**; `target` adalah folder yang dibaca Nginx **dalam container**. `readonly` membuat proses web tidak dapat mengubah file host. Buka `<http://127.0.0.1:8088/>` di Chrome. Di Codespaces, gunakan panel **Ports** untuk membuka 8088 setelah container hidup. Halaman awal harus menampilkan `STATUS_OK: Layanan normal`.

Fungsi/cara kerja validasi: `Test-Path/test -f` memastikan index ada sebelum `docker run`; `Resolve-Path/cd site && pwd` mengambil path folder sebenarnya. Nilai mount ditampilkan agar Anda dapat membaca Source sebelum Docker memakainya. Tanda kutip pada `--mount` menjaga nilai dengan spasi sebagai satu argumen. Jika `$mount` belum dibuat, Docker dapat membaca `nginx:alpine` sebagai argumen mount; menjalankan seluruh blok atau starter mencegah langkah yang terlewat.

Gambar 7. Validasi lokasi repo dan sumber bind mount situs Lab 04

```
docker inspect cloudlab-site --format '{{range .Mounts}}{{.Source}} -> {{.Destination}} RW={{.RW}}{{end}}'
curl.exe -I http://127.0.0.1:8088/
curl -I http://127.0.0.1:8088/
```

Meet 4: mount dari lokasi repo → web HTTP 200

Output command aktual ditata agar terbaca; panel web menampilkan layanan yang sedang berjalan.

Lokasi di bawah adalah contoh pengujian. Starter mencari site/index.html berdasarkan lokasi script, bukan folder terminal.

1. Starter / repair → 2. Verifikasi mount & HTTP

```
$ bash "D:/cloud services/Lab04_Docker_Siap_Push/scripts/start-lab04.sh" --repair
REPO: /d/cloud services/Lab04_Docker_Siap_Push
HTML: /d/cloud services/Lab04_Docker_Siap_Push/site/index.html
MOUNT: type=bind,source=D:/cloud
services/Lab04_Docker_Siap_Push/site,target=/usr/share/nginx/html,readonly
REPAIR: membuat ulang container lab cloudlab-site; file host tetap ada.
cloudlab-site
3de1f0b997bace119dad209022a3496fd4a552dd8ac87021a68946e3219ccd0e
D:/cloud services/Lab04_Docker_Siap_Push/site -> /usr/share/nginx/html RW=false
HTTP 200 OK
WEB: http://127.0.0.1:8088/
Lanjutkan langkah lab dari root repo: /d/cloud services/Lab04_Docker_Siap_Push
$ docker inspect cloudlab-site --format '{{range .Mounts}}{{.Source}} -> {{.Destination}} RW={{.RW}}{{end}}'
$ curl.exe -I http://127.0.0.1:8088/
docker inspect cloudlab-site --format <Source -> Target RW
D:/cloud services/Lab04_Docker_Siap_Push/site -> /usr/share/nginx/html RW=false

curl.exe -I http://127.0.0.1:8088/
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Thu, 08 Oct 2026 07:49:28 GMT
Content-Type: text/html
Content-Length: 1451
Last-Modified: Wed, 07 Oct 2026 11:13:43 GMT
Connection: keep-alive
ETag: "6ac62967-5ab"
Accept-Ranges: bytes
```

Source: folder site dalam repo. Target: web root Nginx. RW=false: mount hanya baca. HTTP 200: halaman berhasil dilayani.

3. Web live — http://127.0.0.1:8088/

CLOUD SERVICES · LAB 04

Status layanan internal

STATUS_OK: Layanan normal

Halaman ini disajikan Nginx dari folder `site/` pada komputer lokal.

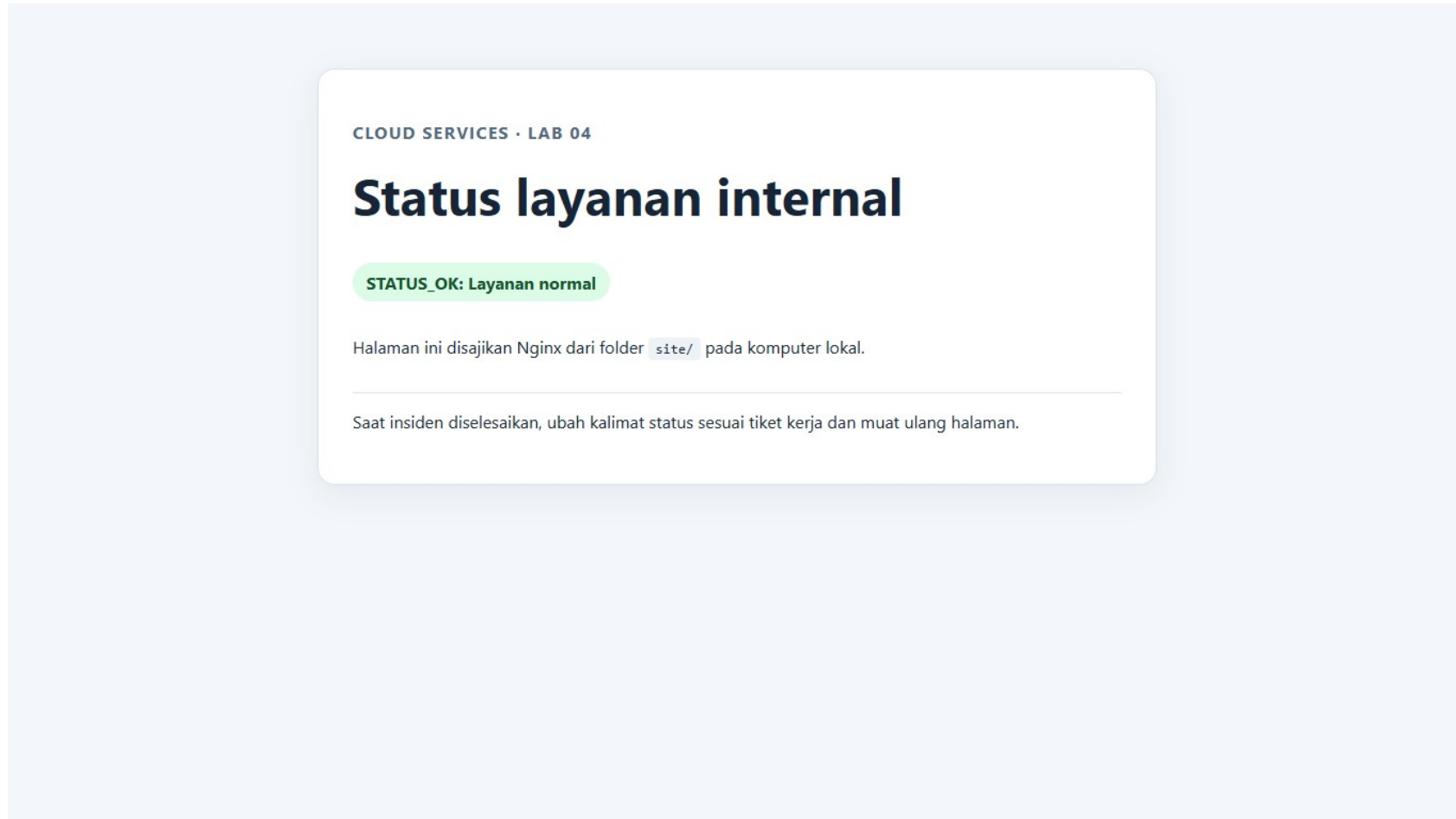
Saat insiden diselesaikan, ubah kalimat status sesuai tiket kerja dan muat ulang halaman.

File yang ditampilkan: `site/index.html`. HTTP 403 sebelumnya dapat terjadi ketika folder induk yang kosong atau salah dimount.

Command / langkah: jalankan starter yang sesuai lokasi terminal atau seluruh blok deploy manual, lalu `docker inspect cloudlab-site --format '{{range .Mounts}}{{.Source}} -> {{.Destination}} RW={{.RW}}{{end}}'` dan `curl.exe -I http://127.0.0.1:8088/` (Bash: `curl -I http://127.0.0.1:8088/`). Fungsi/cara kerja: validasi memastikan index ada; starter menentukan repo dari lokasi script atau blok manual menyelesaikan path dari root repo. Inspect membaca Source yang benar-benar dipakai Docker dan curl meminta header HTTP. Hasil yang diperiksa: Source menunjuk site di repo yang benar, target `/usr/share/nginx/html`, `RW=false`, dan HTTP 200. Path bukti runtime adalah contoh; nama folder pada PC Anda dapat berbeda.

Gambar 8. Halaman status awal melalui container

```
cloudlab-site  
--mount
```



Langkah: jalankan `cloudlab-site` dengan `--mount`, lalu buka port 8088. Fungsi: menerbitkan halaman status internal. Cara kerja: Nginx membaca `site/index.html` langsung dari laptop; browser mencapai Nginx lewat mapping host ke container. Baca hasil: teks status awal dan HTTP 200.

Periksa mount dan port tanpa menebak:

```
docker inspect cloudlab-site --format `
  '{{range .Mounts}}{{.Source}} -> {{.Destination}} RW={{.RW}}{{end}}`
docker port cloudlab-site 80
docker logs --tail 10 cloudlab-site
```

Cari Source folder **site dalam repo**, target `/usr/share/nginx/html` `RW=false`, dan port `127.0.0.1:8088`. `RW=false` adalah bukti mount read-only, bukan klaim dari nama folder. `logs` menunjukkan request yang baru dibuat browser/curl.

Jika situs Up tetapi HTTP 403

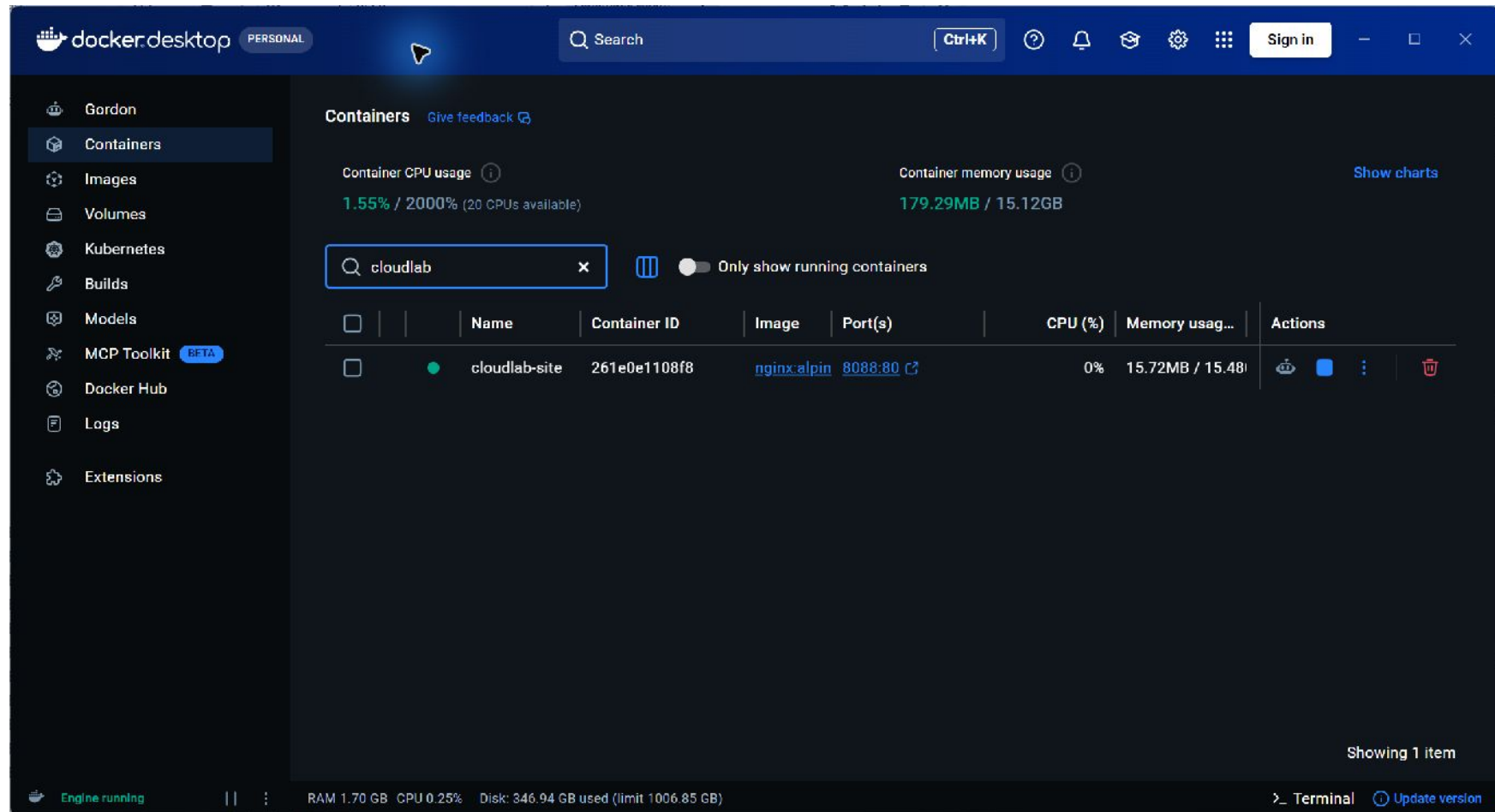
HTTP 403 berarti request sampai ke Nginx tetapi tidak dapat dilayani. Pada kasus folder bersarang, penyebab yang perlu diperiksa dahulu adalah folder host `site/` yang salah/kosong; direktori tanpa index dapat ditolak karena directory listing tidak diaktifkan. Pemeriksaan berikut tidak mengubah file:

```
docker inspect cloudlab-site --format `
  '{{range .Mounts}}{{.Source}} -> {{.Destination}} RW={{.RW}}{{end}}`
docker logs --tail 20 cloudlab-site
docker exec cloudlab-site ls -la /usr/share/nginx/html
```

Cara membaca: cocokkan Source dengan folder repo yang berisi `site/index.html`. `ls` dalam container harus menampilkan `index.html`; log dapat menjelaskan `directory index ... is forbidden` atau kendala izin. Jika Source salah, jalankan **starter dari repo yang benar** dengan `-Repair/--repair`, lalu uji HTTP dan refresh browser. Jangan menyalin HTML ke folder luar agar mount yang salah terlihat berhasil. Jika index/Source sudah benar, lanjutkan pemeriksaan error log karena 403 juga dapat disebabkan konfigurasi/izin; jangan mengubah izin secara acak.

Gambar 9. Docker Desktop memperlihatkan container situs

```
docker inspect  
docker port
```



Langkah: periksa `docker inspect`, `docker port`, dan tampilan Containers di Docker Desktop. Fungsi: memastikan sumber file, mode mount, dan port benar. Cara kerja: `inspect` membaca metadata container yang sedang berjalan; `port` membaca aturan forwarding. Baca hasil: `RW=false`, container Up, dan host 8088 menuju port 80.

4. Challenge kasus kerja A-E

Kerjakan dahulu tanpa melihat kunci bila ingin latihan mandiri, lalu diskusikan bersama dosen. Screenshot boleh disimpan secara opsional pada [hasil/bukti/lab04/](#) sebagai referensi proyek; tidak ada tugas atau penyerahan tambahan.

Bagian	Situasi dan latihan	Bukti yang diamati
A	Tim meminta bukti bahwa image nginx, Python, Node tersedia dan environment variable diterima proses sekali pakai.	<code>image ls</code> , versi, output <code>cloud</code> .
B	Situs internal harus hanya tersedia di laptop pada 8088; file dari host tidak boleh ditulis container.	<code>docker ps</code> , HTTP 200, <code>inspect</code> dengan <code>RW=false</code> .
C	Seseorang menghentikan <code>cloudlab-site</code> . Diagnosis dari <code>ps</code> , <code>ps -a</code> , dan log, pulihkan tanpa membuat image baru.	Kondisi gagal dan sesudah pulih.
D	Setelah insiden, ubah pengumuman menjadi <code>INCIDENT-042: Pemeliharaan selesai</code> tanpa rebuild.	Browser baru, <code>curl</code> , image ID sama, <code>RW=false</code> .
E	Port 8088 sudah dipakai situs utama. Buktikan bentrokan, lalu jalankan canary pada 8089 sambil situs utama tetap hidup.	Error port, dua port HTTP 200, dua container Up.

Kunci C - diagnosis dan pemulihan outage

```
docker stop cloudlab-site
curl.exe -I http://127.0.0.1:8088/
docker ps --filter name=cloudlab-site
docker ps -a --filter name=cloudlab-site
docker logs --tail 10 cloudlab-site
docker start cloudlab-site
curl.exe -I http://127.0.0.1:8088/
```

`curl.exe` pertama **seharusnya gagal** karena container berhenti; itu bukti insiden terkontrol, bukan kegagalan praktikum. `ps` kosong namun `ps -a` menunjukkan `Exited`; log memberi konteks request sebelumnya. `start` menghidupkan container yang sama. `curl.exe` terakhir harus HTTP 200. Dalam pekerjaan nyata, cek state dan log sebelum membangun ulang agar penyebab tidak tertutup.

Gambar 10. Outage teridentifikasi dan layanan dipulihkan

```
ps -a
```

● ● ● C · diagnosis outage dan recovery

```
PS> docker stop cloudlab-site
PS> curl.exe -I http://127.0.0.1:8088/ # expected fail
curl.exe : curl: (7) Failed to connect to 127.0.0.1:8088 after 2033 ms: Could not connect to
server
[exit 7]
PS> docker ps -a --filter name=cloudlab-site
261e0e1108f8 nginx:alpine Exited (0) cloudlab-site
PS> docker start cloudlab-site
PS> curl.exe -I http://127.0.0.1:8088/
HTTP/1.1 200 OK
```

Cuplikan output uji lokal aktual; baris/kolom dipilih dan ditata agar terbaca.

Langkah: stop situs, uji HTTP gagal, baca `ps -a/log`, lalu start. Fungsi: meniru respons insiden layanan. Cara kerja: port 8088 berhenti menerima koneksi saat proses Nginx berhenti; `start` mengaktifkan kembali container yang sama. Baca hasil: `Exited` dan gagal koneksi berubah menjadi `Up` dan HTTP 200. Gambar adalah cuplikan output uji lokal aktual yang ditata agar terbaca.

Gambar 11. Chrome menolak koneksi saat situs berhenti

```
docker stop cloudlab-site  
http://127.0.0.1:8088/
```



This site can't be reached

127.0.0.1 refused to connect.

Try:

- Checking the connection
- [Checking the proxy and the firewall](#)

ERR_CONNECTION_REFUSED

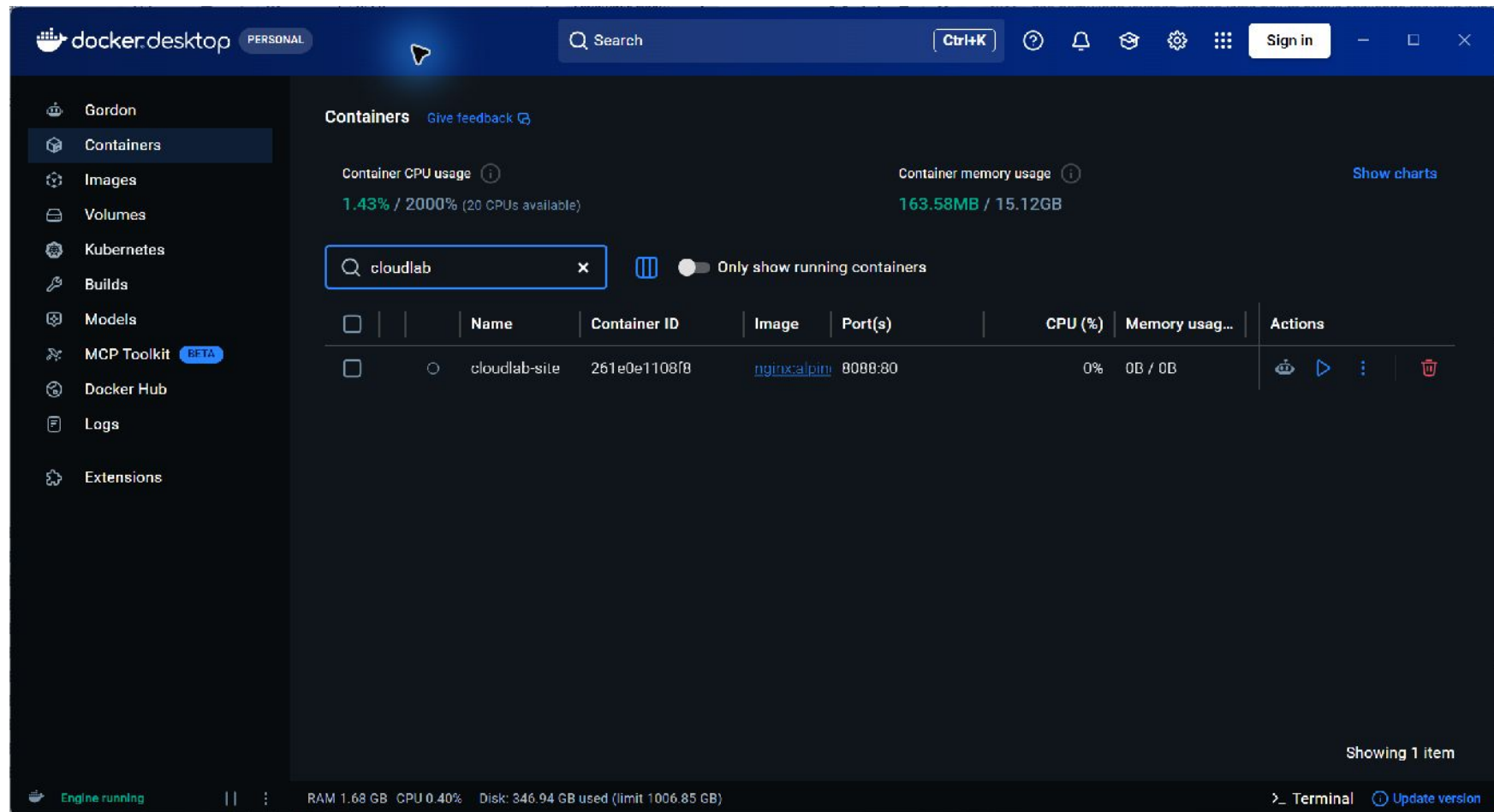
Reload

Details

Langkah: setelah `docker stop cloudlab-site`, refresh `http://127.0.0.1:8088/` di Chrome. Fungsi: melihat gejala yang dialami pengguna. Cara kerja: tidak ada proses Nginx yang menerima koneksi host 8088. Baca hasil: `ERR_CONNECTION_REFUSED`; setelah `docker start`, refresh lagi dan halaman pulih.

Gambar 12. Docker Desktop menunjukkan situs berhenti

```
cloudlab-site  
docker stop
```



Langkah: lihat `cloudlab-site` pada tab Containers setelah `docker stop`. Fungsi: membandingkan gejala browser dengan status container. Cara kerja: Docker Desktop membaca state dari daemon. Baca hasil: indikator container tidak aktif dan tombol Start tersedia; `docker ps -a` memberi status teks `Exited`.

Kunci D - hotfix konten tanpa rebuild

PowerShell - jalankan dari root repo. Perintah penulisan memakai UTF-8 tanpa BOM agar hasil HTTP bersih:

```
$imageBefore = docker image inspect nginx:alpine --format '{{.Id}}'
$sitePath = (Resolve-Path .\site\index.html).Path
$html = [System.IO.File]::ReadAllText($sitePath)
$html = $html.Replace('STATUS_OK: Layanan normal', 'INCIDENT-042: Pemeliharaan selesai')
[System.IO.File]::WriteAllText($sitePath, $html, (New-Object System.Text.UTF8Encoding($false)))
$imageAfter = docker image inspect nginx:alpine --format '{{.Id}}'
$imageBefore -eq $imageAfter
curl.exe -fsS http://127.0.0.1:8088/ | Select-String 'INCIDENT-042'
docker inspect cloudlab-site --format '{{range .Mounts}}{{.Destination}} RW={{.RW}}{{end}}'
```

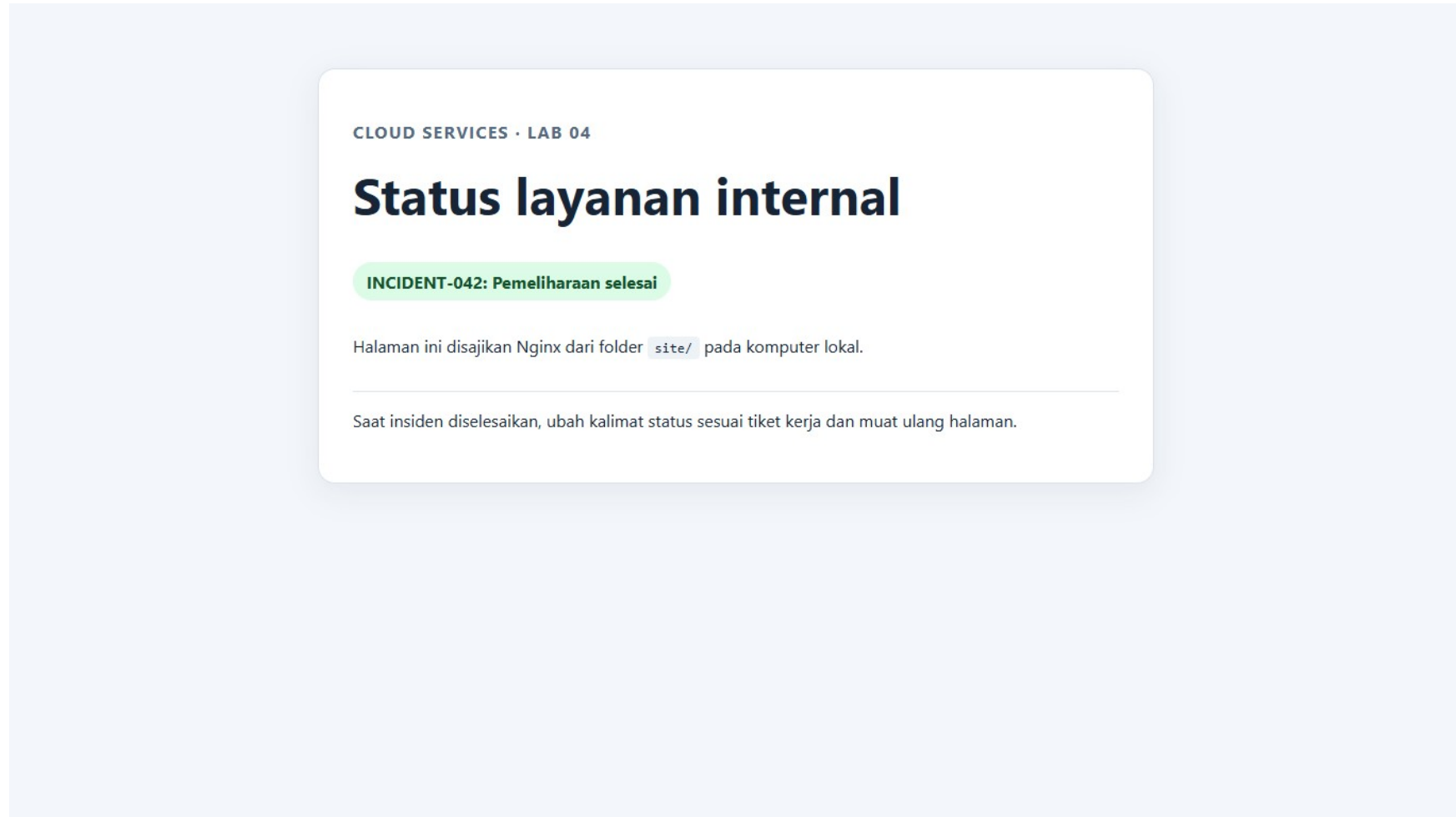
Bash/Linux/Codespaces:

```
before=$(docker image inspect nginx:alpine --format '{{.Id}}')
sed -i 's/STATUS_OK: Layanan normal/INCIDENT-042: Pemeliharaan selesai/' site/index.html
after=$(docker image inspect nginx:alpine --format '{{.Id}}')
test "$before" = "$after" && echo 'image tetap sama'
curl -fsS http://127.0.0.1:8088/ | grep 'INCIDENT-042'
docker inspect cloudlab-site --format '{{range .Mounts}}{{.Destination}} RW={{.RW}}{{end}}'
```

Kamu juga boleh mengganti kalimat melalui editor VS Code lalu menyimpan file. Refresh Chrome. **True** pada PowerShell atau **image tetap sama** pada Bash berarti image tidak dibangun ulang; bind mount menyajikan file host terbaru. **RW=false** tetap menunjukkan Nginx tidak diberi hak tulis ke host.

Gambar 13. Hotfix langsung terlihat di halaman web

```
site/index.html
```



Langkah: ubah satu kalimat di `site/index.html`, simpan, dan refresh Chrome. Fungsi: menyampaikan status insiden tanpa redeploy image. Cara kerja: bind mount membuat Nginx membaca perubahan file host pada request berikutnya. Baca hasil: **INCIDENT-042: Pemeliharaan selesai** muncul, sementara image ID sebelum/sesudah sama.

Kunci E - bentrokan port dan canary

Jalankan perintah pertama saat `cloudlab-site` masih **Up**. Kegagalan port yang muncul adalah hasil yang diharapkan:

```
docker run --name cloudlab-canary -d -p 127.0.0.1:8088:80 nginx:alpine
docker ps -a --filter name=cloudlab-canary
docker rm -f cloudlab-canary
docker run --name cloudlab-canary -d -p 127.0.0.1:8089:80 nginx:alpine
docker ps --filter name=cloudlab-
curl.exe -I http://127.0.0.1:8088/
curl.exe -I http://127.0.0.1:8089/
```

Jika Docker tidak membuat objek canary saat percobaan pertama, `docker rm -f cloudlab-canary` boleh menampilkan `No such container`; lanjutkan. Port host 8088 hanya dapat dimiliki satu proses, tetapi kedua container tetap bisa mendengar port **internal** 80 karena namespace jaringan mereka berbeda. Canary pada host 8089 adalah cara mencoba versi/instans kedua tanpa mematikan layanan utama.

Gambar 14. Bentrokan port 8088 dan solusi 8089

coba port 8088 untuk canary, baca error, bersihkan objek percobaan, lalu jalankan pada 8089.

● ● ● E · port host bentrok, canary pindah ke 8089

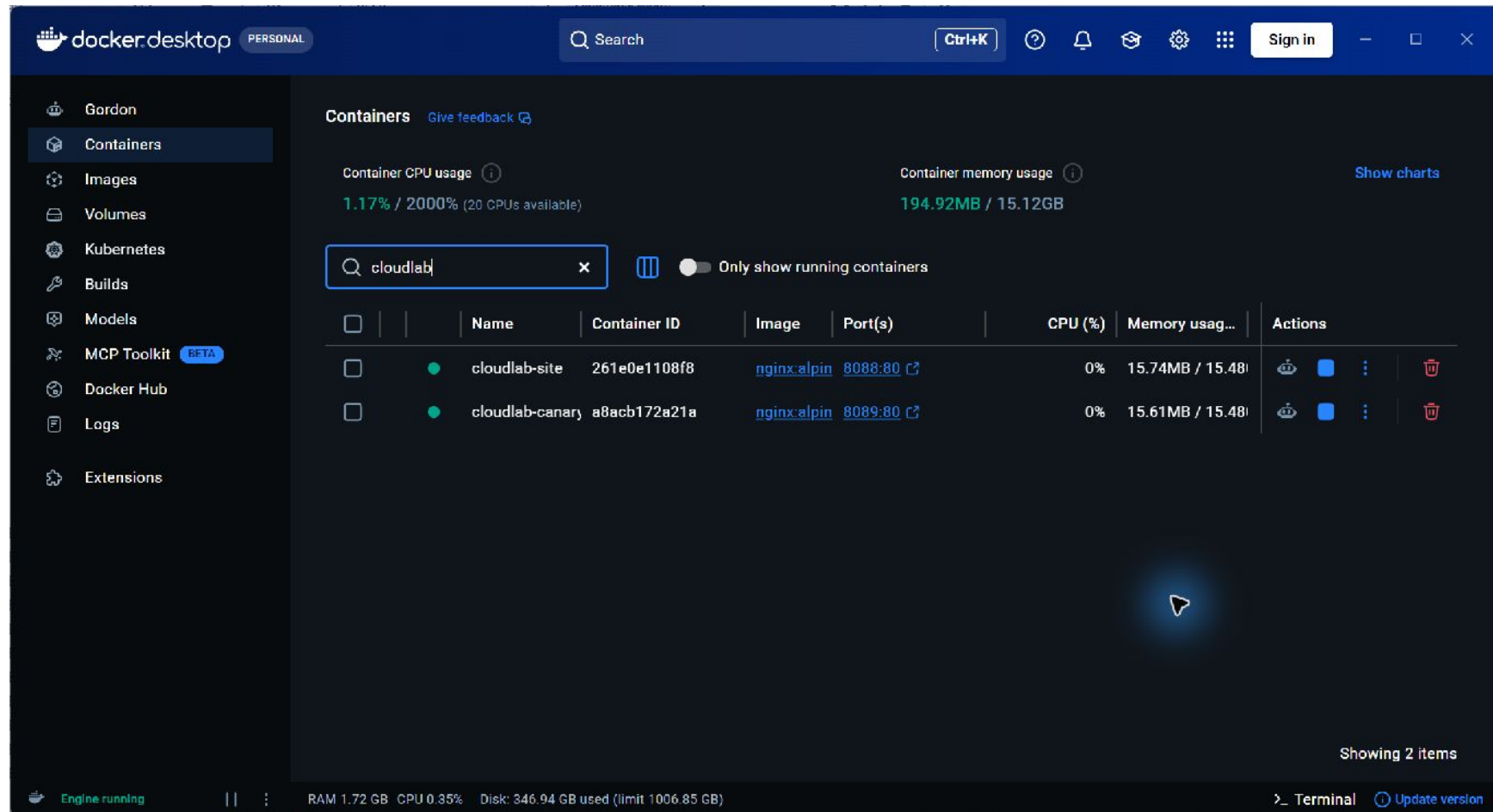
```
PS> docker run cloudlab-canary -p 127.0.0.1:8088:80 # expected conflict
... Bind for 127.0.0.1:8088 failed: port is already allocated
[exit 125]
PS> docker ps -a --filter name=cloudlab-canary
cd32410c1537 cloudlab-canary Created
PS> docker rm -f cloudlab-canary
PS> docker run cloudlab-canary -p 127.0.0.1:8089:80
a8acb172a21a cloudlab-canary Up 127.0.0.1:8089->80/tcp
261e0e1108f8 cloudlab-site Up 127.0.0.1:8088->80/tcp
PS> curl.exe -I http://127.0.0.1:8088/
HTTP/1.1 200 OK
PS> curl.exe -I http://127.0.0.1:8089/
HTTP/1.1 200 OK
```

Cuplikan output uji lokal aktual; baris/kolom dipilih dan ditata agar terbaca.

Langkah: coba port 8088 untuk canary, baca error, bersihkan objek percobaan, lalu jalankan pada 8089. Fungsi: melatih diagnosis `port is already allocated`. Cara kerja: host tidak dapat mengikat 8088 dua kali; port container 80 tetap boleh sama. Baca hasil: kedua container Up dan HTTP 200 pada 8088 serta 8089. Gambar adalah cuplikan output uji lokal aktual yang ditata agar terbaca.

Gambar 15. Dua container hidup di Docker Desktop pada port berbeda

lihat tab Containers sesudah menjalankan canary pada 8089.



Langkah: lihat tab Containers sesudah menjalankan canary pada 8089. Fungsi: memeriksa bahwa situs utama tetap hidup saat instance kedua ditambah. Cara kerja: Docker Desktop menampilkan dua container dan dua pemetaan port host menuju port 80 masing-masing. Baca hasil: `cloudlab-site` 8088:80 dan `cloudlab-canary` 8089:80 sama-sama aktif.

Kunci A dan B - pemeriksaan akhir

Untuk A, gunakan perintah pada bagian 1. Untuk B, gunakan `docker ps`, `docker port`, `docker inspect`, dan `curl` pada bagian 3. Keduanya tidak membutuhkan kode aplikasi baru. Urutan A-E yang lengkap diuji oleh skrip berikut dari **root repo** setelah hotfix dan canary hidup:

PowerShell:

```
& .\tests\challenge.ps1
```

Bash/Linux/Codespaces/Git Bash:

```
bash tests/challenge.sh
```

Jika terminal masih berada di folder induk, panggil `& .\meet4CloudService\tests\challenge.ps1` atau `bash "meet4CloudService/tests/challenge.sh"`. Dari lokasi lain, panggil path lengkap checker dalam tanda kutip. Checker menentukan root dari lokasi script; untuk command manual yang mengedit `site/index.html`, gunakan terminal pada root repo seperti bagian 3.

Tes memeriksa hasil akhir yang dapat diamati: target fungsi **14 PASS, 0 FAIL**, bukan nilai praktikum. Environment variable A, riwayat stop/start C, dan percobaan bentrokan E dapat diamati saat demo; bila ingin merekam proses, simpan screenshot/catatan opsional karena status akhir tidak membuktikan kejadian itu pernah dilakukan. Pada Windows, jalankan skrip Bash dari **jendela Git Bash**, bukan mengetik `bash` dalam PowerShell yang mungkin membuka WSL.

Gambar 16. Challenge A-E lulus setelah semua perbaikan

```
tests/challenge.ps1
tests/challenge.sh
```



```
Challenge A-E · PowerShell

PS> & .\tests\challenge.ps1
PASS A image nginx:alpine tersedia
PASS A image python:3.12-alpine tersedia
PASS A image node:22-alpine tersedia
PASS B cloudlab-site berjalan dari nginx:alpine
PASS B port 8088 hanya di 127.0.0.1
PASS B folder site/ di-bind mount read-only
PASS C halaman utama HTTP 200 di 8088
PASS D halaman menampilkan INCIDENT-042: Pemeliharaan selesai
PASS D status lama sudah diganti
PASS E cloudlab-canary berjalan dari nginx:alpine
PASS E port 8089 hanya di 127.0.0.1
PASS E canary HTTP 200 di 8089
PASS E cloudlab-site tidak membuka port host lain
PASS E cloudlab-canary tidak membuka port host lain

Challenge: 14 PASS, 0 FAIL

Cuplikan output uji lokal aktual; baris/kolom dipilih dan ditata agar terbaca.
```

Langkah: jalankan `tests/challenge.ps1` atau `tests/challenge.sh`. Fungsi: mengecek image, situs, mount, HTTP, canary, dan port akhir. Cara kerja: skrip membaca Docker metadata dan melakukan request lokal; ia memberi PASS/FAIL tanpa memperbaiki konfigurasi. Baca hasil: **Challenge: 14 PASS, 0 FAIL**. Gambar adalah cuplikan output uji lokal aktual; output pribadi boleh disimpan sebagai referensi proyek.

5. Catatan opsional, Git, dan cleanup

Jika ingin menyimpan catatan pribadi, salin [templat laporan](#) ke `hasil/lab04.md` dan isi dengan penjelasan serta screenshot di `hasil/bukti/lab04/`. Ini opsional, tanpa penyerahan atau nilai lab terpisah. Jangan menyimpan output `docker info` penuh. Blok Git berikut hanya digunakan bila file catatan/bukti tersebut sudah dibuat **pada repo milik Anda**, dengan origin pribadi yang diperiksa melalui `git remote -v`; clone publik pengajar cukup dipakai lokal.

```
git status --short
git add site/index.html hasil/lab04.md hasil/bukti/lab04
git diff --cached --check
git diff --cached --name-only
git commit -m "lab04: pulihkan situs dan dokumentasikan insiden"
git push
```

`git status` menunjukkan perubahan, `git add` memilih catatan opsional, `diff --cached --check` memeriksa whitespace, dan `push` mengirim commit ke origin pribadi. Jangan push ke repo pengajar atau commit data host, kredensial, maupun screenshot dosen sebagai bukti sendiri. Tidak perlu push image ke registry. Setelah praktik selesai, lanjutkan cleanup container lab di bawah, termasuk jika tidak membuat catatan/commit.

Gambar 17. Repo template Lab 04 setelah commit dan push

```
git diff --cached --check
git push
```

The screenshot shows a GitHub repository named 'meet4CloudService' created by user 'budijarto'. It is a public template repository. The main branch contains a commit titled 'lab04: add Docker lifecycle case lab and complete modules' made 3 hours ago. The repository includes several folders and files, all committed at the same time. The right-hand sidebar provides summary statistics: 0 stars, 0 forks, and no releases or packages published yet.

Langkah: jalankan `git diff --cached --check` sebelum commit, lalu `git push` setelah commit pada repo milik Anda. Fungsi: memeriksa whitespace pada perubahan yang dipilih dan mengunggah commit bila memilih menyimpan kontribusi proyek di GitHub. Cara kerja: `diff --cached --check` membaca staging area dan melaporkan whitespace bermasalah; `push` mengirim commit lokal ke remote origin. Baca hasil: buka repo pribadi, periksa branch main, pesan commit terbaru, serta catatan/screenshot pribadi. Foto menunjukkan repo template dosen sebagai contoh lokasi commit; nama dan isi repo peserta akan berbeda. Langkah ini opsional dan tidak mewajibkan penyerahan Lab 04.

```
docker rm -f cloudlab-canary cloudlab-site
docker ps -a --filter name=cloudlab-
```

Image tetap ada sehingga dapat dipakai lagi tanpa pull ulang. `rm` menghapus container, sedangkan file `site/index.html` tetap ada pada host dan di Git.

Checkpoint cleanup: `docker ps -a --filter name=cloudlab-` hanya menampilkan header tanpa baris container lab. Buka tab **Containers** di Docker Desktop untuk memastikan `cloudlab-site` dan `cloudlab-canary` sudah tidak ada. Gambar pada langkah 3 dan 4 memperlihatkan kondisi sebelum cleanup sehingga perbedaannya dapat dibandingkan.

Jika ada kendala

Gejala	Pemeriksaan dan perbaikan
Cannot connect to Docker daemon	Buka Docker Desktop hingga Engine running; ulangi <code>docker version</code> .
Port 8088/8089 sudah dipakai	<code>docker ps</code> untuk mencari container lab. Jangan menghentikan container lain tanpa tahu pemiliknya; gunakan port kosong dan sesuaikan command, URL, serta catatan opsional bila perlu.
Nama container sudah dipakai	<code>docker ps -a --filter name=cloudlab-</code> ; jika sisa percobaan milikmu, <code>docker rm -f</code> nama tersebut, lalu ulangi.
Error <code>--mount</code> menyebut <code>nginx:alpine</code>	<code>\$mount</code> mungkin kosong/belum dibuat pada terminal itu. Jalankan starter atau seluruh blok bagian 3 termasuk inisialisasi variabel pada sesi yang sama.
<code>--mount</code> gagal di path berspasi	Periksa index/root dengan validasi bagian 3; gunakan tanda kutip pada seluruh nilai <code>--mount</code> atau jalankan starter dari lokasi script yang benar.
Nginx Up tetapi HTTP 403	Bandingkan Source mount, log, dan <code>ls</code> dalam container pada bagian 3. Pastikan index.html berasal dari site dalam repo, bukan site luar yang kosong; starter repair dapat memasang ulang folder yang benar.
Starter menolak nama container yang ada	Tinjau nama/image. Mode repair hanya untuk cloudlab-site/cloudlab-nginx Nginx yang dikenali; jangan menghapus container lain untuk memaksa berhasil.
Halaman tidak berubah	Pastikan <code>site/index.html</code> tersimpan, mount mengarah ke folder repo yang benar, lalu refresh Chrome.
Challenge C/D/E gagal	Periksa <code>docker ps</code> , <code>docker port</code> , <code>docker inspect</code> , dan isi <code>site/index.html</code> ; baca hint dari skrip sebelum mengulang.
PowerShell menolak <code>.ps1</code>	Jalankan <code>bash tests/challenge.sh</code> dari Git Bash/WSL, atau minta dosen membantu dengan kebijakan kampus; langkah Docker tetap dapat dilakukan di PowerShell.